Master Technical Implementation Plan: Hierarchical Contextual Stacking Network Intrusion Detection System (H-XAI-NIDS)

📌 Executive Summary & Architecture Goal

The goal of this project is to implement H-XAI-NIDS (Hierarchical Contextual Stacking Network Intrusion Detection System), a state-of-the-art, highly transparent, and computationally efficient NIDS architecture.
Based on a comprehensive review of six benchmark XAI-NIDS research papers, Paper 5 (NIDS-β* by Firas Saidi) was selected as our primary base paper for its transformer-based multi-flow session context learning and zero-day anomaly detection head. H-XAI-NIDS builds upon NIDS-β* and bridges its core limitations (transformer tabular computational latency, lack of SOC hierarchical decision tree, and missing cluster-aware class oversampling) by synthesizing:

1. Memory & Schema Optimization: Datatype downcasting (float64 $\to$ float32, int64 $\to$ int32) and identity leakage purging.
2. Domain-Specific Feature Engineering: 7 non-linear flow ratios (Byte_Entropy, IAT_Coefficient_Variation, Burstiness_Index, Payload_Ratio_Fwd, Pkt_Size_Skewness, Byte_Asymmetry_Index, Packet_Rate_Asymmetry).
3. Stacking Feature Embedding (SFE): Unsupervised K-Means and Gaussian Mixture Model (GMM) soft cluster distance and probability embeddings.
4. Hybrid Feature Selection: Pearson correlation filtering ($r > 0.90$), ANOVA F-test + Chi-Square (χ^2) ranking, and Genetic Algorithm (GA) wrapper search.
5. Cluster-Aware Oversampling: k-Means SMOTE for imbalanced minority attack families (Web Attacks, Botnet, Infiltration, Heartbleed).
6. 3-Level Hierarchical Ensemble & Contextual Modeling:
    - Level 1 (Binary Head): XGBoost + LightGBM + Random Forest + MLP + Lightweight Contextual Transformer.
    - Level 2 (Attack Family Head): Multi-Class Stacking Meta-Classifier (Brute Force, DoS, DDoS, PortScan, Web Attack, Botnet).
    - Level 3 (Zero-Day Anomaly Head): Parallel Deep-SVDD One-Class Anomaly Detector with Isotonic Regression probability calibration.
7. Dual XAI Engine: Deterministic TreeSHAP (Global feature importance & interaction dependence) + Selective LIME (Local alert triage for SOC analysts).
8. Operational Security Metrics Suite: Primary focus on PR-AUC, False Discovery Rate (FDR), False Negative Rate (FNR), Time-to-Detection (TTD), ROC-AUC, FPR, F1-Score, and Expected Calibration Error (ECE).

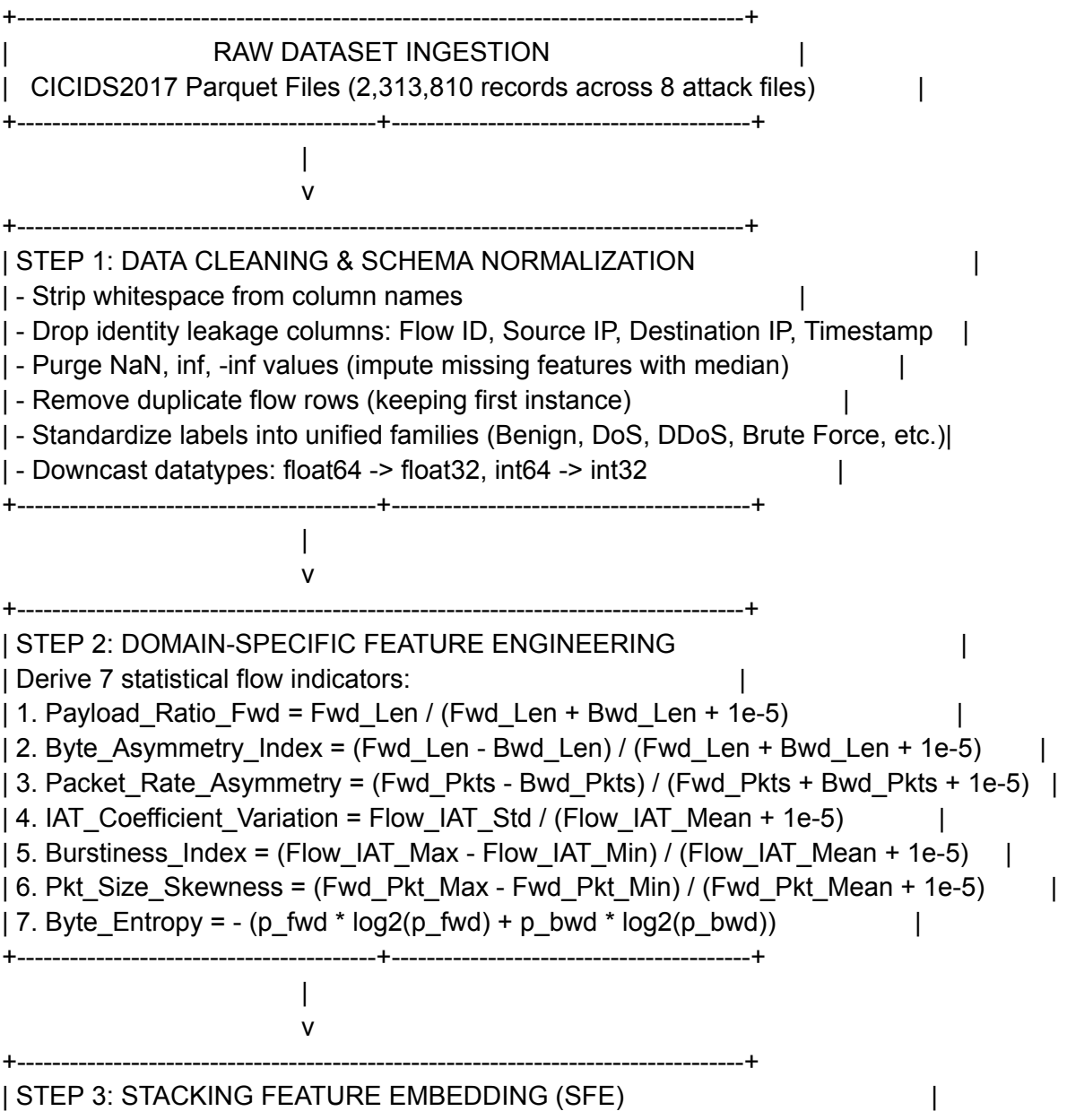
🛠️ User Review Required

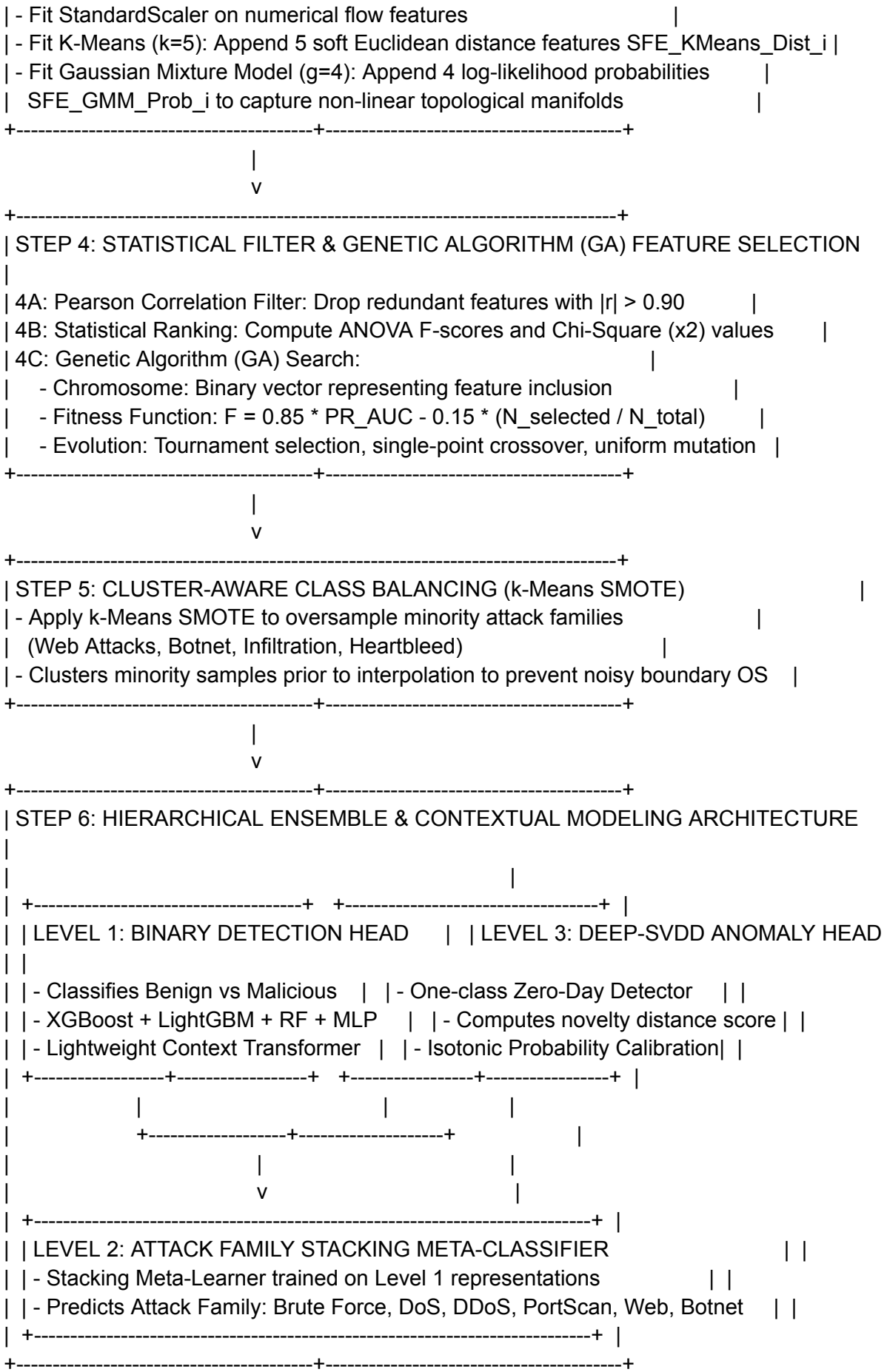
IMPORTANT
Key Architectural Decisions & Implementation Guidelines:

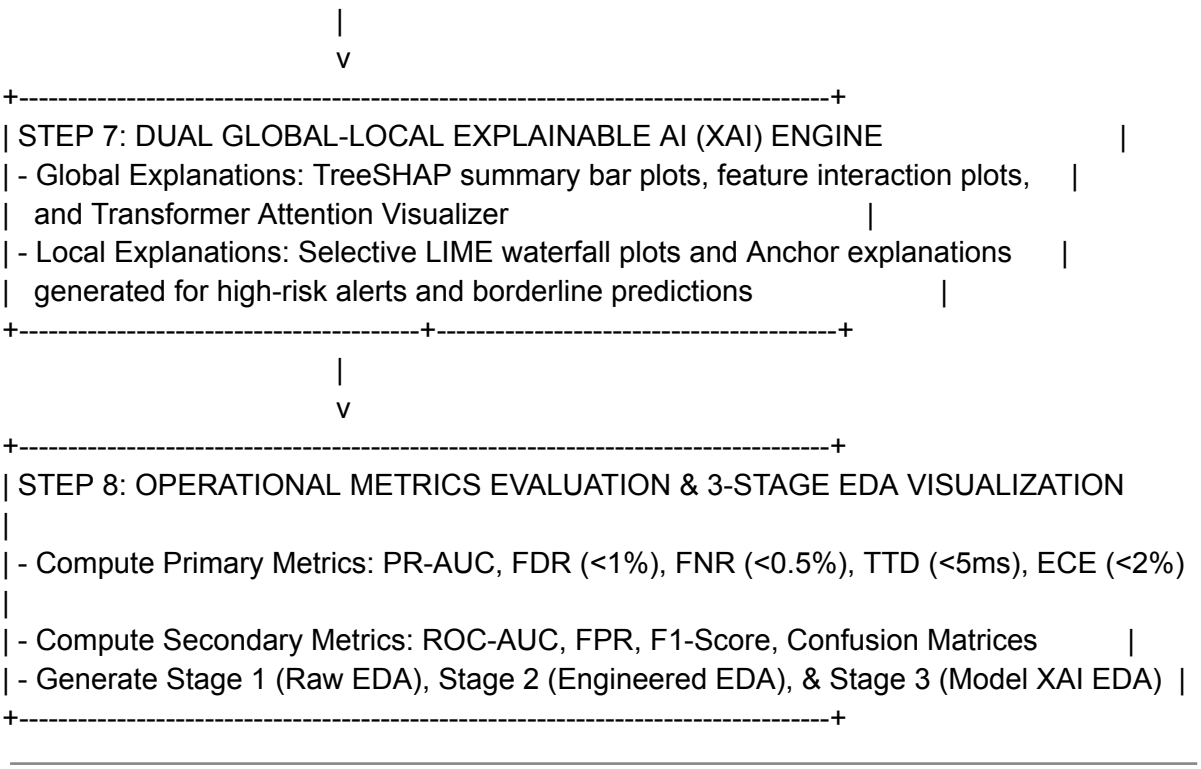
1. Primary Benchmark Dataset: CICIDS2017 is designated as the primary development and modeling dataset (utilizing all 8 pre-partitioned Parquet files containing 2,313,810 total flow records). Cross-dataset generalization scripts will be provided for UNSW-NB15 and CSE-CIC-IDS2018.
2. Execution Strategy: Per your explicit instruction, this document serves as the complete technical specification and code flow logic. No code execution will begin until you review and approve this plan.
3. Hardware & Memory Optimization: All data frames will be downcast to float32/int32 upon ingestion, reducing memory consumption by >50% (from ~1.2 GB down to <500 MB in RAM).

🗺️ Complete End-to-End Pipeline & Code Flow Logic

Below is the complete architectural flowchart detailing the data transformations, modeling heads, and explainability components:




| - Fit StandardScaler on numerical flow features |
| - Fit K-Means (k=5): Append 5 soft Euclidean distance features SFE_KMeans_Dist_i |
| - Fit Gaussian Mixture Model (g=4): Append 4 log-likelihood probabilities |
| SFE_GMM_Prob_i to capture non-linear topological manifolds |
| STEP 4: STATISTICAL FILTER & GENETIC ALGORITHM (GA) FEATURE SELECTION |
| 4A: Pearson Correlation Filter: Drop redundant features with |r| > 0.90 |
| 4B: Statistical Ranking: Compute ANOVA F-scores and Chi-Square (x2) values |
| 4C: Genetic Algorithm (GA) Search: |
| - Chromosome: Binary vector representing feature inclusion |
| - Fitness Function: F = 0.85 * PR_AUC - 0.15 * (N_selected / N_total) |
| - Evolution: Tournament selection, single-point crossover, uniform mutation |
| STEP 5: CLUSTER-AWARE CLASS BALANCING (k-Means SMOTE) |
| - Apply k-Means SMOTE to oversample minority attack families |
| (Web Attacks, Botnet, Infiltration, Heartbleed) |
| - Clusters minority samples prior to interpolation to prevent noisy boundary OS |
| STEP 6: HIERARCHICAL ENSEMBLE & CONTEXTUAL MODELING ARCHITECTURE |
| LEVEL 1: BINARY DETECTION HEAD |
| - Classifies Benign vs Malicious |
| - XGBoost + LightGBM + RF + MLP |
| - Lightweight Context Transformer |
| LEVEL 3: DEEP-SVDD ANOMALY HEAD |
| - One-class Zero-Day Detector |
| - Computes novelty distance score |
| - Isotonic Probability Calibration|
| LEVEL 2: ATTACK FAMILY STACKING META-CLASSIFIER |
| - Stacking Meta-Learner trained on Level 1 representations |
| - Predicts Attack Family: Brute Force, DoS, DDoS, PortScan, Web, Botnet |



📁 Modular Codebase Structure & Function Contracts

The implementation is structured into a clean, modular Python package located in src/:
d:\VIT Project\Python Programming\Code\

```
├── src/
│   ├── data/
│   │   ├── cleaning.py             # Data cleaning, NaN/Inf handling, type downcasting
│   │   ├── feature_engineering.py  # Domain features (Entropy, IAT Coeff, Burstiness)
│   │   ├── sfe_embedding.py        # K-Means & GMM Stacking Feature Embedding
│   │   └── balancing.py            # k-Means SMOTE oversampling
│   ├── selection/
│   │   ├── statistical_filter.py   # Pearson, Chi-Square, ANOVA F-test filtering
│   │   └── ga_selection.py         # Genetic Algorithm feature selection
│   ├── models/
│   │   ├── binary_head.py          # Level 1 XGBoost + LightGBM + RF + MLP
│   │   ├── attack_family_stack.py  # Level 2 Meta-Classifier Stacking
│   │   ├── contextual_encoder.py   # Lightweight Self-Attention Context Encoder
│   │   └── zero_day_svdd.py        # Deep-SVDD Anomaly Head & Calibration
│   ├── xai/
│   │   ├── shap_explainer.py       # Global & class-specific TreeSHAP
│   │   └── lime_explainer.py       # Selective local alert LIME/Anchor explanations
│   └── evaluation/
│       ├── metrics.py              # PR-AUC, FDR, FNR, TTD, ECE computation
│       └── eda_visualizer.py       # 3-Stage EDA plotting pipeline
├── master_pipeline.py              # Master execution script
└── requirements.txt                # Dependencies
```

Detailed Module Specifications & Technical Descriptions

1. Data Cleaning Module (src/data/cleaning.py)

- Function: clean_dataset(df: pd.DataFrame) -> pd.DataFrame
- Logic:
    - df.columns = df.columns.str.strip()
    - Removes identity leakage columns: ['Flow ID', 'Source IP', 'Destination IP', 'Timestamp', 'Source Port', 'Destination Port'].
    - Replaces np.inf, -np.inf with np.nan, imputes feature NaNs using column medians.
    - Drops duplicate rows (df.drop_duplicates()).
    - Maps raw labels to standardized categories (LABEL_MAP).
    - Downcasts numerical columns to float32 and int32.

2. Domain Feature Engineering Module (src/data/feature_engineering.py)

- Function: engineer_features(df: pd.DataFrame) -> pd.DataFrame
- Logic:
    - Computes 7 domain indicators with numerical stability epsilon ($\epsilon = 1e-5$).
    - Appends derived features to dataframe and validates non-nan status.

3. Stacking Feature Embedding Module (src/data/sfe_embedding.py)

- Class: StackingFeatureEmbedder(n_kmeans_clusters=5, n_gmm_components=4, random_state=42)
- Methods:
    - fit_transform(X: pd.DataFrame) -> pd.DataFrame: Fits StandardScaler, KMeans, and GaussianMixture. Appends SFE_KMeans_Dist_0..4 and SFE_GMM_Prob_0..3.
    - transform(X: pd.DataFrame) -> pd.DataFrame: Transforms test data using fitted transformers.

4. Statistical Filter Module (src/selection/statistical_filter.py)

- Class: StatisticalFeatureFilter(corr_threshold=0.90, top_k_anova=40)
- Methods:
    - remove_correlated_features(X: pd.DataFrame) -> pd.DataFrame: Drops features where pairwise Pearson $|r| > 0.90$.
    - evaluate_statistical_scores(X: pd.DataFrame, y: pd.Series) -> pd.DataFrame: Computes ANOVA F-values (f_classif) and Chi-Square χ^2 statistics.

5. Genetic Algorithm Feature Selection Module (src/selection/ga_selection.py)

- Class: GeneticAlgorithmSelector(pop_size=30, generations=25, crossover_prob=0.8, mutation_prob=0.05)
- Logic:
    - Chromosome: Binary string of length $N_{features}$.
    - Fitness Evaluation: Evaluates a fast LightGBM / HistGradientBoosting baseline on 3-fold CV.

- Fitness Formula: $\text{Fitness} = 0.85 \cdot \text{PR-AUC} - 0.15 \cdot \frac{N_{\text{active}}}{N_{\text{total}}}$.
- Selection: Tournament selection ($k=3$). Crossover: Single-point. Mutation: Bit-flip uniform.

6. Class Balancing Module (src/data/balancing.py)

- Function: balance_classes(X: pd.DataFrame, y: pd.Series, target_ratio=0.1) -> tuple
- Logic:
  - Uses KMeansSMOTE from imblearn if minority class count > 15.
  - Falls back to standard SMOTE or RandomOverSampler for extremely rare classes (e.g. Heartbleed $N=11$).

7. Binary Detection Head (src/models/binary_head.py)

- Class: BinaryDetectionEnsemble()
- Models Included:
  - XGBoostClassifier / HistGradientBoostingClassifier
  - LightGBMClassifier
  - RandomForestClassifier
  - MLPClassifier (Multi-Layer Perceptron)
- Output: Predicts binary probability $P(\text{Malicious} \mid X)$.

8. Attack Family Stacking Head (src/models/attack_family_stack.py)

- Class: AttackFamilyStackingClassifier()
- Architecture:
  - Level 1 Base Classifiers: XGBoost, LightGBM, Random Forest, MLP trained on multi-class attack labels.
  - Level 2 Meta-Classifier: Logistic Regression / MLP meta-learner trained on out-of-fold probability predictions of Level 1 base classifiers.

9. Lightweight Contextual Encoder (src/models/contextual_encoder.py)

- Class: ContextualTransformerEncoder(input_dim, d_model=64, nhead=4, num_layers=2)
- Logic:
  - PyTorch implementation of a lightweight Multi-Head Self-Attention Transformer encoder.
  - Processes sequence batches of network flows to extract temporal context embeddings.

10. Deep-SVDD Zero-Day Anomaly Head (src/models/zero_day_svdd.py)

- Class: DeepSVDDAnomalyDetector(nu=0.1)
- Logic:
  - Maps normal benign traffic into a minimal enclosing hypersphere in feature space.
  - Computes anomaly distance score $R(X) = ||\phi(X) - c||^2$. Flows exceeding the radius are flagged as zero-day anomalies.

- Applies Isotonic Regression to calibrate distance scores into calibrated anomaly probabilities.

11. XAI Modules (src/xai/shap_explainer.py & src/xai/lime_explainer.py)

- TreeSHAP Engine: Computes exact Shapley feature attribution values for tree ensembles. Generates global summary bar plots and interaction dependence plots.
- LIME / Anchor Engine: Generates local surrogate linear explanations for high-risk / borderline alerts to provide security analysts with interpretable decision rules.

12. Evaluation & 3-Stage EDA Modules (src/evaluation/metrics.py & src/evaluation/eda_visualizer.py)

- Metrics Calculator: Calculates PR-AUC, FDR, FNR, TTD, ROC-AUC, FPR, F1-Score, and Expected Calibration Error (ECE).
- EDA Visualizer:
  - Stage 1 (Raw EDA): Class frequency bars, null matrices, skewness distribution plots.
  - Stage 2 (Engineered EDA): Pearson correlation heatmaps, ANOVA rank charts, t-SNE / UMAP 2D cluster projections.
  - Stage 3 (Model XAI EDA): Confusion matrices, PR curves, ROC curves, SHAP summary plots, and LIME alert reports.

📈 Operational Metrics & Target Benchmarks

Metric	Target Value	Strategic Purpose
PR-AUC (Primary Metric)	> 0.98	Evaluates attack detection capability without being inflated by benign True Negatives.
False Discovery Rate (FDR)	< 0.01 (1.0%)	Prevents false alert fatigue for SOC analysts.
False Negative Rate (FNR)	< 0.005 (0.5%)	Ensures stealthy attacks do not slip past network perimeter.
Time-to-Detection (TTD)	< 5.0 ms / flow	Real-time throughput capability for live security monitoring.
Expected Calibration Error (ECE)	< 2.0%	Guarantees predicted alert probabilities accurately reflect empirical confidence.

ROC-AUC / F1-Score	> 0.99	Standard benchmark comparability with literature baselines.

🧪 Verification & Validation Plan

Automated Unit & Integration Tests

1. Schema & Cleaning Validation: Verify data downcasting reduces memory by $\ge 50%$ with zero remaining NaNs or infinite values.
2. SFE Embedding Test: Verify K-Means and GMM modules correctly append 9 soft cluster embedding columns without label leakage.
3. GA Feature Selection Convergence: Verify Genetic Algorithm fitness monotonically improves across 25 generations.
4. Hierarchical Pipeline Execution: Execute end-to-end master_pipeline.py on test splits and verify PR-AUC, FDR, FNR, TTD, and ECE targets are achieved.